OpsPilot AI - Sprint 4: SQLite Persistent Storage

Goal: Save incidents permanently so they remain available after the backend restarts.

What We Built

- Added SQLAlchemy.
- Created SQLite connection and session handling.
- Created the incidents database table model.
- Replaced temporary RAM storage with database operations.
- Confirmed data survives a Uvicorn restart.

Flow

```
POST alert -> Analyse -> Create database record -> Save in SQLite -> Restart server -> Retrieve same incident
```

requirements.txt

```
fastapi
uvicorn[standard]
sqlalchemy
```

database.py

```
DATABASE_URL = "sqlite:///./data/opspilot.db"

engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)

class Base(DeclarativeBase):
  pass
```

Engine connects Python to SQLite. **SessionLocal** creates sessions for reading and writing. **Base** is the parent for database table models.

models.py

```
class Incident(Base):
  __tablename__ = "incidents"
  alert_id = mapped_column(String, primary_key=True)
  status = mapped_column(String)
  received_at = mapped_column(String)
  alert_type = mapped_column(String)
  server = mapped_column(String)
  value = mapped_column(Float)
  threshold = mapped_column(Float)
  severity = mapped_column(String)
  investigation_status = mapped_column(String)
  probable_cause = mapped_column(String)
  recommended_action = mapped_column(String)
  confidence = mapped_column(Integer)
```

Key Concepts

Concept	Meaning
SQLite	Lightweight database stored in one local file.
SQLAlchemy	Python library for relational database operations.
Model	Python class representing a database table.
Column	A field in the table, such as server or severity.
Primary key	Unique identifier for one row; alert_id is the primary key.
Persistent storage	Data remains after the application stops or restarts.

Creating the Table

```
from app import models
from app.database import Base, engine

Base.metadata.create_all(bind=engine)
```

This creates the incidents table only when it does not already exist.

Database Storage Logic

save_incident(): opens a session, creates an Incident object, adds it, commits, and closes the session.
get_all_incidents(): selects all rows and converts them into dictionaries.
get_incident(): retrieves one row using its alert ID.
try/finally: guarantees that the database session is closed.

Persistence Test

Test	Result
Create HIGH_MEMORY alert	Saved in data/opspilot.db.
GET /incidents	Incident returned.
Stop and restart Uvicorn	Application restarted.
GET /incidents again	Same incident still returned.

Important Git Note

The local database is excluded by **data/*.db** in .gitignore. Code is uploaded to GitHub, but local incident records are not.

Judge Explanation

"We replaced temporary memory storage with SQLite persistence using SQLAlchemy. Incidents are stored as structured database records and remain available after application restarts. This provides reliable incident history while keeping the prototype free and locally runnable."

Quick Questions

Why SQLite? It is free, lightweight, and ideal for a local hackathon prototype.
Why SQLAlchemy? It gives a clean Python interface and makes future migration easier.
What is a primary key? A unique field identifying one record.
Why commit? Commit permanently saves changes.
Why close sessions? To release database resources.
Can SQLite be replaced? Yes, SQLAlchemy supports migration to PostgreSQL or another relational database.

Sprint 4 Summary

OpsPilot AI now stores incidents permanently in SQLite and retrieves them even after the backend restarts.